

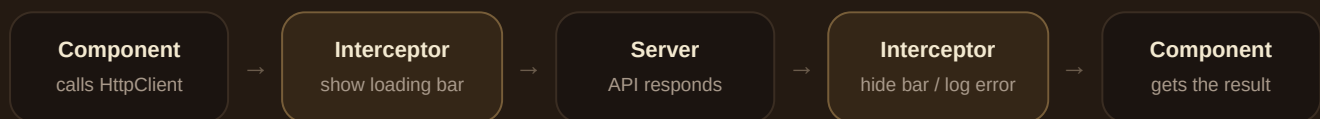
HttpInterceptor

One checkpoint that **every outgoing HTTP request and incoming response** passes through — think of it as airport security for HTTP calls: one place for loading state, auth headers, and error logging. This guide also covers `catchError` vs. `retry`, the two tools you'll add to a real interceptor.

1 · EXPLAINED SIMPLY

Every request passes through the same checkpoint

An interceptor sits between your code and the network. It can inspect or modify the outgoing request, and inspect or modify the response (or error) on the way back — for every single HTTP call, without touching each component individually.



2 · THE CODE

A functional `HttpInterceptorFn`

```
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);
  loadingService.show();

  return next(req).pipe(
    catchError((error) => {
      console.error(`Request to ${req.url} failed:`, error.message);
      return throwError(() => error);
    }),
    finalize(() => loadingService.hide())
  );
};
```

- `show()` / `hide()` just increment / decrement a counter signal — `LoadingService.isLoading()` is true while it's above zero.
- `finalize()` runs whether the request succeeds, fails, or is cancelled by `switchMap` — the loading bar never gets stuck on.
- Registered once: `provideHttpClient(withInterceptors([loadingInterceptor]))` in `app.config.ts`.

`src/app/core/interceptors/loading.interceptor.ts`

3 · TWO DIFFERENT TOOLS

catchError vs. retry

`catchError` and `retry` solve different problems — knowing which to reach for matters.

CATCHERROR	RETRY
Use when: the error is final — show a message, log it, or fall back to a default value.	Use when: the failure might be transient — a flaky connection, a momentary server hiccup.
Runs exactly once per error.	<code>retry({ count: 2, delay: 500 })</code> re-subscribes to the source up to 2 more times before giving up.
Comes after <code>retry</code> in the pipe, so it only sees the final failure.	Goes before <code>catchError</code> in the pipe — transient failures get a second chance first.

4 · PUTTING IT TOGETHER

Where each piece sits in the pipe

```
return next(req).pipe(
  retry({ count: 2, delay: 500 }),      // try transient failures again first
  catchError((error) => {              // then handle whatever is still failing
    console.error('Request failed:', error.message);
    return throwError(() => error);     // rethrow – components can still react
  }),
  finalize(() => loadingService.hide()) // always runs: success, error, or cancel
);
```

Order matters: `retry` closest to the source, `catchError` after it, `finalize` last so cleanup always runs. This is a direct extension of the loading interceptor above — `retry` is the one addition worth adding once transient failures start showing up in your logs.